

AI-Powered Transaction Processing Pipeline

A five-stage banking pipeline, built end to end by a workflow of specialised AI agents.

Andrii Chernenko

Capstone — Homework 6

12.08.2026

THE PROBLEM

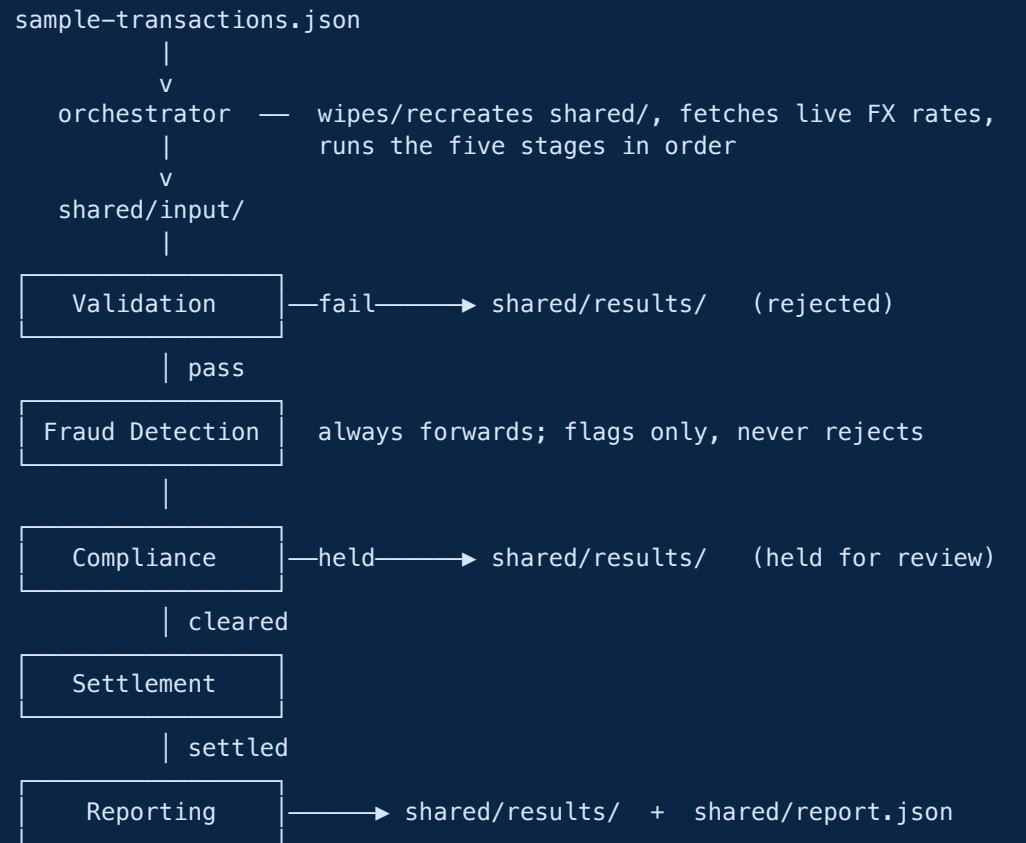
What the system does

The pipeline ingests raw financial transaction records and carries every one of them through a fixed sequence of stages that **validate**, **risk-score**, **compliance-screen**, **settle** and **report** on it.

A record that fails a check stops there and is finalised with a reason; only a transaction clearing every stage is settled. Nothing is silently dropped — every transaction ends with an auditable outcome.

Run state and results are published as plain JSON as the run proceeds, so any external process can observe progress without parsing logs or reading pipeline code.

Flow through the shared file tree



shared/status.json is updated live as each stage starts and completes.

Stage responsibilities

- **Validation** — checks required fields and types, that amount parses as a precise decimal, and that currency is a real ISO 4217 code. Also runs standalone as a non-mutating dry check that writes nothing.
- **Fraud Detection** — scores three weighted, explainable factors: high value, cross-border mismatch, unusual-hour timing. Flags at 0.50 and above but never rejects.
- **Compliance Check** — screens against a GDPR / Data Protection Act 2018 baseline. A flagged transaction is *held for human review* rather than auto-rejected, per GDPR Article 22.
- **Settlement** — settles cleared transactions against an internal simulated ledger, issuing a settlement reference, timestamp and retention period. Refuses anything not compliance-cleared.
- **Reporting** — aggregates outcomes into a run summary, then joins each transaction with every result it accumulated into a final record.

Five agents built this

Agent	Responsibility
specification	Writes the technical spec from a shared template. Never explores the repo; unresolved gaps become explicit clarification markers rather than guesses.
pipeline-codegen	Generates the orchestrator and five stage modules from the spec, researching the stack through context7 before writing a line.
skills-hooks	Reproduces the workflow slash commands and the coverage-gate settings.
tests-codegen	Generates the unit and integration suites and measures coverage.
documentation	Discovers the stack at run time and writes README and HOWTORUN from what it finds — nothing hardcoded.

A single slash command, **/run-complete-flow**, runs all five in a fixed order and rebuilds the project from scratch.

Skills, hooks and MCP

/run-pipeline

Runs the pipeline end to end and summarises every transaction that did not settle, with the stage and reason that stopped it.

/validate-transactions

Validates the whole dataset without processing it and without writing anything.

Coverage gate hook

A PreToolUse hook that intercepts `git push` and blocks it whenever coverage falls below 80%.

MCP servers

context7 for live library documentation during code generation; a custom **pipeline-status** FastMCP server exposing read-only transaction status and results.

Every skill is stack-agnostic: none names a language, runtime or endpoint — each discovers them from the project on every run.

DEMO

Pipeline run

```
andriichern@MacBook-Air-Andrii:~/Projects/gen-ai-software-engineering/homework-6$ python3 orchestrator.py
Wiping and recreating shared/ ...
Copying input dataset from sample-transactions.json into shared/input/ ...
Ingested 8 transaction record(s).
Fetching live exchange rates for currencies: ['EUR', 'GBP', 'USD'] ...
Exchange rates fetched: {'USD': '1', 'EUR': '0.86509', 'GBP': '0.74062'}

=== Stage: validation ===
[AUDIT] timestamp=2026-08-11T19:17:44.692614+00:00 stage=validation transaction_id=TXN001 outcome=passed
[AUDIT] timestamp=2026-08-11T19:17:44.693402+00:00 stage=validation transaction_id=TXN002 outcome=passed
[AUDIT] timestamp=2026-08-11T19:17:44.694058+00:00 stage=validation transaction_id=TXN003 outcome=passed
[AUDIT] timestamp=2026-08-11T19:17:44.694630+00:00 stage=validation transaction_id=TXN004 outcome=passed
[AUDIT] timestamp=2026-08-11T19:17:44.695233+00:00 stage=validation transaction_id=TXN005 outcome=passed
[AUDIT] timestamp=2026-08-11T19:17:44.695893+00:00 stage=validation transaction_id=TXN006 outcome=failed: invalid currency code: XYZ
[AUDIT] timestamp=2026-08-11T19:17:44.696546+00:00 stage=validation transaction_id=TXN007 outcome=passed
[AUDIT] timestamp=2026-08-11T19:17:44.697105+00:00 stage=validation transaction_id=TXN008 outcome=passed
validation: processed=8 passed=7 failed=1

=== Stage: fraud_detection ===
[AUDIT] timestamp=2026-08-11T19:17:44.698246+00:00 stage=fraud_detection transaction_id=TXN001 outcome=not_flagged
[AUDIT] timestamp=2026-08-11T19:17:44.698731+00:00 stage=fraud_detection transaction_id=TXN002 outcome=flagged
[AUDIT] timestamp=2026-08-11T19:17:44.699291+00:00 stage=fraud_detection transaction_id=TXN003 outcome=not_flagged
[AUDIT] timestamp=2026-08-11T19:17:44.699763+00:00 stage=fraud_detection transaction_id=TXN004 outcome=flagged
[AUDIT] timestamp=2026-08-11T19:17:44.700228+00:00 stage=fraud_detection transaction_id=TXN005 outcome=flagged
[AUDIT] timestamp=2026-08-11T19:17:44.700706+00:00 stage=fraud_detection transaction_id=TXN007 outcome=not_flagged
[AUDIT] timestamp=2026-08-11T19:17:44.701206+00:00 stage=fraud_detection transaction_id=TXN008 outcome=not_flagged
fraud_detection: processed=7 passed=4 failed=3

=== Stage: compliance ===
[AUDIT] timestamp=2026-08-11T19:17:44.702193+00:00 stage=compliance transaction_id=TXN001 outcome=cleared
[AUDIT] timestamp=2026-08-11T19:17:44.702694+00:00 stage=compliance transaction_id=TXN002 outcome=held
[AUDIT] timestamp=2026-08-11T19:17:44.703162+00:00 stage=compliance transaction_id=TXN003 outcome=cleared
[AUDIT] timestamp=2026-08-11T19:17:44.703647+00:00 stage=compliance transaction_id=TXN004 outcome=held
[AUDIT] timestamp=2026-08-11T19:17:44.704083+00:00 stage=compliance transaction_id=TXN005 outcome=held
[AUDIT] timestamp=2026-08-11T19:17:44.704442+00:00 stage=compliance transaction_id=TXN007 outcome=cleared
[AUDIT] timestamp=2026-08-11T19:17:44.704787+00:00 stage=compliance transaction_id=TXN008 outcome=cleared
compliance: processed=7 passed=4 failed=3

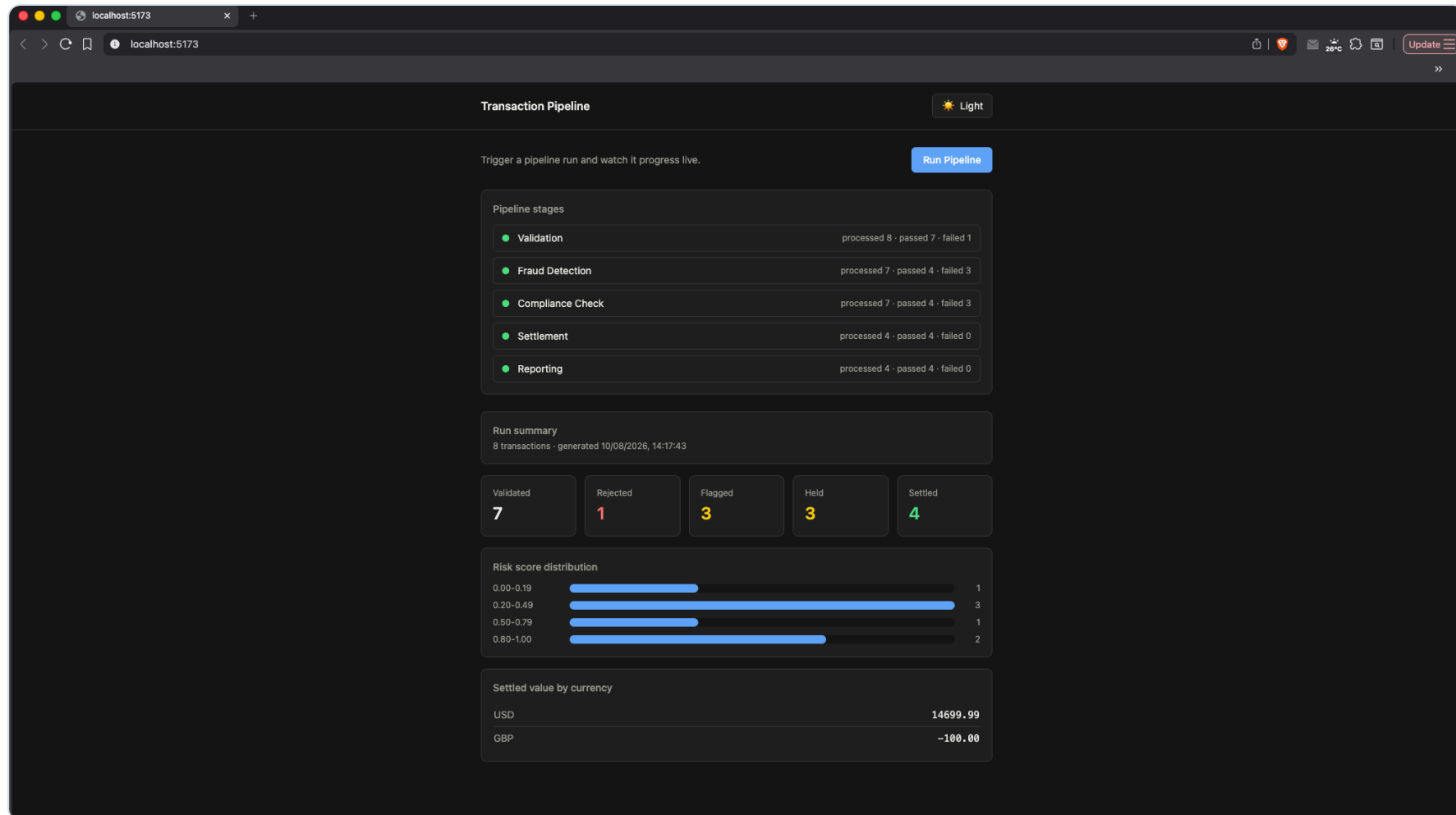
=== Stage: settlement ===
[AUDIT] timestamp=2026-08-11T19:17:44.705551+00:00 stage=settlement transaction_id=TXN001 outcome=settled
[AUDIT] timestamp=2026-08-11T19:17:44.705915+00:00 stage=settlement transaction_id=TXN003 outcome=settled
[AUDIT] timestamp=2026-08-11T19:17:44.706280+00:00 stage=settlement transaction_id=TXN007 outcome=settled
[AUDIT] timestamp=2026-08-11T19:17:44.706671+00:00 stage=settlement transaction_id=TXN008 outcome=settled
settlement: processed=4 passed=4 failed=0

=== Stage: reporting ===
[AUDIT] timestamp=2026-08-11T19:17:44.708548+00:00 stage=reporting transaction_id=TXN001 outcome=settled
[AUDIT] timestamp=2026-08-11T19:17:44.708792+00:00 stage=reporting transaction_id=TXN003 outcome=settled
[AUDIT] timestamp=2026-08-11T19:17:44.709022+00:00 stage=reporting transaction_id=TXN007 outcome=settled
[AUDIT] timestamp=2026-08-11T19:17:44.709245+00:00 stage=reporting transaction_id=TXN008 outcome=settled
reporting: processed=4 passed=4 failed=0
Clearing shared/input/ ...

Run complete. Results in shared/results/, summary at shared/report.json.
andriichern@MacBook-Air-Andrii:~/Projects/gen-ai-software-engineering/homework-6$
```

DEMO

Front-end dashboard



DEMO

Skill in action — /run-pipeline

Run header:

- Entrypoint: python3 orchestrator.py
- Input dataset: sample-transactions.json
- Records in final-outcome directory: 8 of 8

Counts by final status:

Final Status	Count
settled	4
held	3
rejected	1

Per-transaction table:

Transaction	Final Status	Stopping Stage
TXN001	settled	-
TXN002	held	compliance
TXN003	settled	-
TXN004	held	compliance
TXN005	held	compliance
TXN006	rejected	validation
TXN007	settled	-
TXN008	settled	-

Transactions that did not settle:

Validation (rejected):

- TXN006: Invalid currency code XYZ - rejected before entering fraud detection

Compliance (held for human review):

- TXN002: Fraud score 0.80 flagged this record - held at compliance stage
- TXN004: Fraud score 0.50 flagged this record - held at compliance stage
- TXN005: Fraud score 0.80 flagged this record - held at compliance stage

Rollup from report.json:

- Total records processed: 8
- Validated (passed validation): 7
- Flagged by fraud detection: 3
- Held for compliance review: 3
- Successfully settled: 4
- Risk score distribution: 0.00-0.10 (1), 0.20-0.49 (3), 0.50-0.79 (1), 0.80-1.00 (2)
- Total settled value by currency: USD \$14,699.99 | GBP -£100.00

Record count verification: ✓ All 8 input records reached the final-outcome directory. Input and output counts match.

* Crunched for 42s

ssh scripts/check-coverage.sh

manual mode on - 7 for shortcuts - e for agents

QUALITY

Tests and coverage

100%

statement & branch coverage

528

of 528 statements covered

318

test functions

80%

enforced gate

```
andrichern@MacBook-Air-Andrii:~/Projects/gen-ai-software-engineering/homework-6
└─ _nghomework-6 (-zsh) X1
└─ homework-6 (-zsh) X2

tests/test_validation.py::TestValidateTransaction::test_missing_source_account PASSED [ 90%]
tests/test_validation.py::TestValidateTransaction::test_missing_destination_account PASSED [ 91%]
tests/test_validation.py::TestValidateTransaction::test_missing_amount PASSED [ 91%]
tests/test_validation.py::TestValidateTransaction::test_missing_currency PASSED [ 91%]
tests/test_validation.py::TestValidateTransaction::test_missing_transaction_type PASSED [ 91%]
tests/test_validation.py::TestValidateTransaction::test_missing_description PASSED [ 92%]
tests/test_validation.py::TestValidateTransaction::test_missing_metadata PASSED [ 92%]
tests/test_validation.py::TestValidateTransaction::test_invalid_transaction_id_type PASSED [ 92%]
tests/test_validation.py::TestValidateTransaction::test_invalid_timestamp_format PASSED [ 93%]
tests/test_validation.py::TestValidateTransaction::test_invalid_timestamp_type PASSED [ 93%]
tests/test_validation.py::TestValidateTransaction::test_invalid_source_account_type PASSED [ 93%]
tests/test_validation.py::TestValidateTransaction::test_invalid_destination_account_type PASSED [ 94%]
tests/test_validation.py::TestValidateTransaction::test_amount_must_be_string PASSED [ 94%]
tests/test_validation.py::TestValidateTransaction::test_invalid_amount_format PASSED [ 94%]
tests/test_validation.py::TestValidateTransaction::test_invalid_currency_code PASSED [ 94%]
tests/test_validation.py::TestValidateTransaction::test_invalid_transaction_type_type PASSED [ 95%]
tests/test_validation.py::TestValidateTransaction::test_invalid_description_type PASSED [ 95%]
tests/test_validation.py::TestValidateTransaction::test_metadata_not_dict PASSED [ 95%]
tests/test_validation.py::TestValidateTransaction::test_missing_metadata_channel PASSED [ 96%]
tests/test_validation.py::TestValidateTransaction::test_missing_metadata_country PASSED [ 96%]
tests/test_validation.py::TestValidateTransaction::test_empty_metadata_channel PASSED [ 96%]
tests/test_validation.py::TestValidateTransaction::test_empty_metadata_country PASSED [ 97%]
tests/test_validation.py::TestValidateTransaction::test_null_metadata_channel PASSED [ 97%]
tests/test_validation.py::TestValidateTransaction::test_null_metadata_country PASSED [ 97%]
tests/test_validation.py::TestValidateTransaction::test_valid_transactions_from_fixture PASSED [ 97%]
tests/test_validation.py::TestValidateTransaction::test_invalid_transactions_from_fixture PASSED [ 98%]
tests/test_validation.py::TestValidateTransaction::test_edge_case_transactions_from_fixture PASSED [ 98%]
tests/test_validation.py::TestValidateTransaction::test_negative_amount_passes PASSED [ 98%]
tests/test_validation.py::TestValidateTransaction::test_very_large_amount_passes PASSED [ 99%]
tests/test_validation.py::TestValidateTransaction::test_zero_amount_passes PASSED [ 99%]
tests/test_validation.py::TestValidateTransaction::test_multiple_currencies PASSED [ 99%]
tests/test_validation.py::TestValidateTransaction::test_decimal_precision PASSED [100%]

===== tests coverage =====
coverage: platform darwin, python 3.9.5-final-0

Name                               Stats    Miss Branch BrPart  Cover    Missing
-----
lib/__init__.py                      0      0      0      0  100.00%
lib/common.py                       72      0     10      0  100.00%
lib/exchange_rates.py                32      0     10      0  100.00%
lib/stage_runner.py                  21      0      8      0  100.00%
orchestrator.py                      83      0     16      0  100.00%
pipeline/__init__.py                  0      0      0      0  100.00%
pipeline/compliance.py               38      0      4      0  100.00%
pipeline/fraud_detection.py           72      0     12      0  100.00%
pipeline/reporting.py                 75      0     22      0  100.00%
pipeline/settlement.py                42      0      2      0  100.00%
pipeline/validation.py                76      0     28      0  100.00%
TOTAL                               511      0    112      0  100.00%

Coverage HTML written to dir htmlcov
Coverage JSON written to file .coverage_raw.json

===== 327 passed in 7.86s =====

Building coverage_report.json ...

Test Summary
-----
Tests: PASSED
Coverage: 100.0% >= 80% threshold

Reports:
- Structured coverage: coverage_report.json
- Browseable coverage: htmlcov/index.html
andrichern@MacBook-Air-Andrii:~/Projects/gen-ai-software-engineering/homework-6$ s homework-6-submission.s
```

QUALITY

The gate actually fires

The screenshot shows a VS Code editor with a project named 'homework-6'. The file explorer on the left shows a directory structure with various test files and configuration files. The terminal window, titled 'Bash tool output (uifuab)', shows the output of a git push command. The output indicates that the coverage gate failed because the current coverage (69.18%) is below the threshold (80%). The chat window on the right, titled 'CHAT CLAUDE CODE', provides a detailed explanation of the hook's logic and the reason for the failed push.

```
1 Exit code 1
2
3
4 Coverage Gate Check
5
6 ✓ Found coverage report: coverage_report.json
7 ✓ Detected format: json
8
9 Coverage Metrics:
10 Actual: 69.18%
11 Threshold: 80%
12
13 x Coverage gate FAILED: 69.18% < 80%
14
15 x Push blocked. Please run tests and improve coverage before pushing.
16 Command: bash run_tests.sh
17
18 error: failed to push some refs to 'github.com:andriichern/gen-ai-software-engineering.git'
```

Switch to Claude Opus model

then add git hook and make the script smart (should be triggered only for homework-6 folder). does claude hook need any changes?

All three paths use the same smart script that only enforces the gate for homework-6.

Claude hook needs no changes — it already references the same script correctly.

push current branch as is

- Thought for 1s >
- Attempting push to test both hooks:
- Bash**
 - IN git push 2>61
 - OUT Exit code 1
- Thought for 2s >
- Hook is working perfectly!**
The git pre-push hook triggered and blocked the push:
 - ✓ Found coverage report: coverage_report.json
 - ✓ Detected format: json
 - Coverage Metrics:
 - Actual: 69.18%
 - Threshold: 80%
 - x Coverage gate FAILED: 69.18% < 80%
 - x Push blocked. Please run tests and improve coverage before p
- What happened:**
 1. ✓ Git hook detected the push attempt
 2. ✓ Called check-coverage.sh
 3. ✓ Found coverage report (69.18%)
 4. ✓ Compared to threshold (80%)
 5. ✓ Blocked push because coverage < threshold

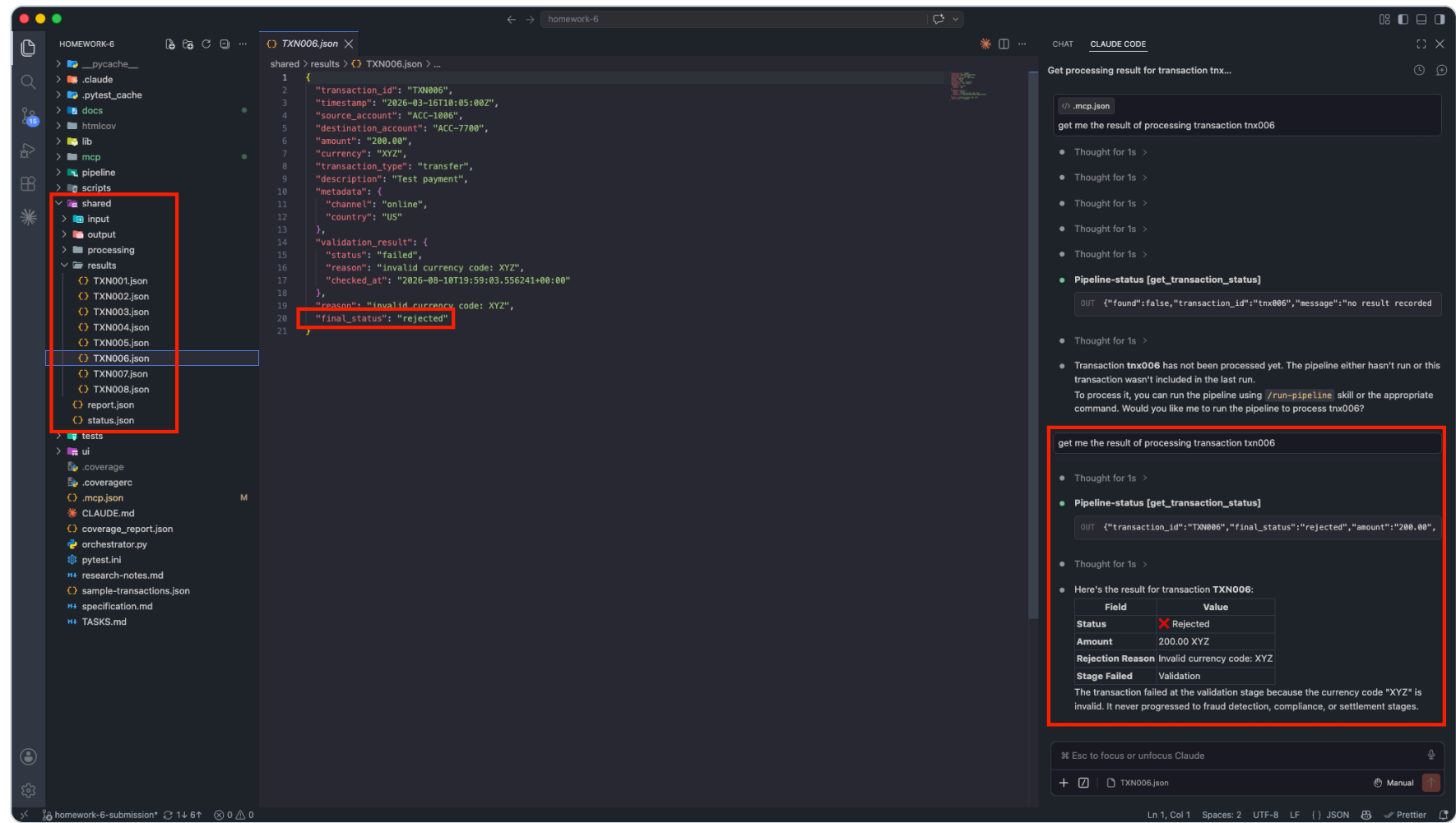
Exit code: 1 (push rejected)

⌘ Esc to focus or unfocus Claude

+ Manual

Ln 1, Col 1 Spaces: 2 Plain Text Prettier

MCP in use



Lessons learned

- **A nested agent cannot ask you anything.** An agent spawned by another agent silently loses its question tool — even when its own definition declares it. Any agent that must ask has to be dispatched from the top level, which decided the whole shape of the orchestrating command.
- **Models diverge exactly where the context is thin.** Given the same underspecified instruction, different models fill the gap differently — one asks, one picks a sensible default, one invents something plausible and states it as fact. The variation is not in capability but in what each treats as permission to decide.
- **So the rules must be strict, explicit and absolute.** Every agent here says what it must never do, not merely what it should do — never assume, never infer from what happens to exist, never resolve a gap silently, and an exhaustive allowlist of what it may write. Ambiguity is where behaviour stops being reproducible.
- **Scope fences have to be written many times over.** A restriction stated in several places must be changed in every one of them at once; a single missed line leaves an agent holding contradictory instructions, which it will resolve in whichever direction you did not expect.
- **Destructive steps and interactive steps must be ordered carefully.** A blind cleanup that runs before the user is asked what to build can destroy work the run was never going to rebuild.
- **Verify the platform, don't assume it.** Two design decisions here rested on assumptions about agent behaviour that a one-minute probe proved wrong — one in each direction.